

GaussianNLLLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.GaussianNLLLoss.html>

Description:

Gaussian negative log likelihood loss. The targets are treated as samples from Gaussian distributions with expectations and variances predicted by the neural network. For a target tensor modelled as having Gaussian distribution with a tensor of expectations input and a tensor of positive variances var the loss is: $-\frac{1}{2} \log \frac{1}{\det \Sigma} - \frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)$ where Σ is the covariance matrix. By default, the constant term of the loss function is omitted unless full is True. If var is not the same size as input (due to a homoscedastic assumption), it must either have a final dimension of 1 or have one fewer dimension (with all other sizes being the same) for correct broadcasting. full (bool, optional) – include the constant term in the loss calculation. Default: False. eps (float, optional) – value used to clamp var (see note below), for stability. Default: 1e-6.

Function Signature

```
class torch.nn.GaussianNLLLoss(*, full=False, eps=1e-06,  
reduction='mean')[source]#
```

Parameters

Shape:

Reference:

```
forward(input, target, var)[source]#
```

Returns:

Tensor

Code Examples

```
>>> loss = nn.GaussianNLLLoss()
>>> input = torch.randn(5, 2, requires_grad=True)
>>> target = torch.randn(5, 2)
>>> var = torch.ones(5, 2, requires_grad=True) #
heteroscedastic
>>> output = loss(input, target, var)
>>> output.backward()
```

```
>>> loss = nn.GaussianNLLLoss()
>>> input = torch.randn(5, 2, requires_grad=True)
>>> target = torch.randn(5, 2)
>>> var = torch.ones(5, 1, requires_grad=True) # homoscedastic
>>> output = loss(input, target, var)
>>> output.backward()
```

Notes & Warnings

NOTE Note The clamping of var is ignored with respect to autograd, and so the gradients are unaffected by it.

Detailed Documentation

Documentation

Gaussian negative log likelihood loss.

The targets are treated as samples from Gaussian distributions with expectations and variances predicted by the neural network. For a target tensor modelled as having Gaussian distribution with a tensor of expectations input and a tensor of positive variances var the loss is:

where eps is used for stability. By default, the constant term of the loss function is omitted unless full is True. If var is not the same size as input (due to a homoscedastic assumption), it must either have a final dimension of 1 or have one fewer dimension (with all other sizes being the same) for correct broadcasting.

full (bool, optional) – include the constant term in the loss calculation. Default: False.

eps (float, optional) – value used to clamp var (see note below), for stability. Default: 1e-6.

reduction (str, optional) – specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the output is the average of all batch member losses, 'sum': the output is the sum of all batch member losses. Default: 'mean'.

Input: (N,*)(N,*)(N,*) or (*)(*)(*) where * means any number of additional dimensions

Target: (N,*)(N,*)(N,*) or (*)(*)(*), same shape as the input, or same shape as the input but with one dimension equal to 1 (to allow for broadcasting)

Var: (N,*)(N,*)(N,*) or (*)(*)(*), same shape as the input, or same shape as the input but with one dimension equal to 1, or same shape as the input but with one fewer dimension (to allow for broadcasting), or a scalar value

Output: scalar if reduction is 'mean' (default) or 'sum'. If reduction is 'none', then $(N,*)$ $(N, *)$ $(N,*)$, same shape as the input

The clamping of var is ignored with respect to autograd, and so the gradients are unaffected by it.

Nix, D. A. and Weigend, A. S., “Estimating the mean and variance of the target probability distribution”, Proceedings of 1994 IEEE International Conference on Neural Networks (ICNN’94), Orlando, FL, USA, 1994, pp. 55-60 vol.1, doi: 10.1109/ICNN.1994.374138.

Runs the forward pass.